Finite-State Verification of UGV Navigation Decisions

1 Architecture and assumptions

The UGV controller has five persistent supervisor modes: INIT, NAVIGATE, HOLD_GUARD, HOLD_FAULT, and GOAL. Only NAVIGATE permits the existing path follower; all other modes request zero linear and angular velocity. The existing full pose and route guards remain authoritative. Memory includes a fault latch, a clear-decision counter, recovery age, and completed-goal memory. The completion latch prevents estimated drift or intervening holds from restarting the same goal. It records an estimated-threshold observation, not R2's independent physical/quiet dwell. The supplied decision-precedence diagram is available separately as `figures/supervisor.pdf`. The detector boundary is optional and disabled in the camera deployment because no online producer is available. When enabled, missing, stale, invalid, or discontinuous decisions hold. Recovery requires ten distinct valid clear decisions and at least five simulation seconds from the first clear acquisition stamp, with fresh latest evidence. New goals never clear fault recovery.

2 Formal method and implementation

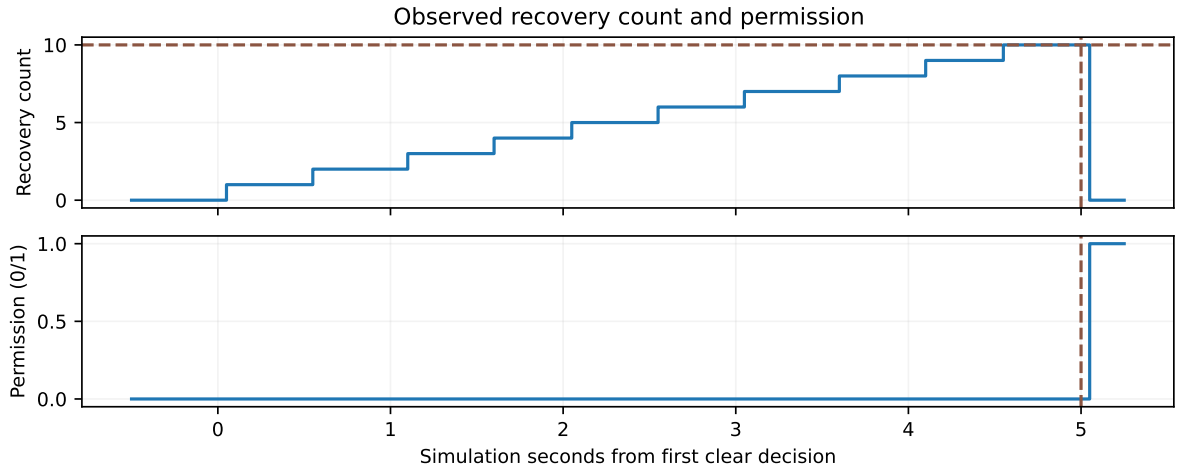
A versioned decision table generates C++ mode-selection data and supplies a Python reference model. Independent temporal safety and next-step predicates constrain permission, completion, resets, readiness, and exact recovery-memory updates. An independently maintained completion monitor is composed with the reachable-state graph. Examples include

$$G(\text{permit} \rightarrow \text{goalValid} \wedge \text{poseValid} \wedge \text{routeValid} \wedge \neg\text{faultLatch} \wedge \neg\text{done})$$

and $G(\text{done} \wedge \neg\text{newGoal} \rightarrow X\,\text{done})$. Breadth-first exploration includes all latch/counter memory and recovery-age classes; it is exhaustive finite-state safety checking rather than only mode/input enumeration. Exact numerical tests establish timestamp/freshness boundaries, including five seconds plus/minus one nanosecond. The pure C++ transition and validators have no ROS/Gazebo imports. Every received detector decision updates recovery, including alarm/clear bursts. The ROS controller reevaluates permission before periodic command publication. Policy and implementation digests bind the compiled probe to the checked sources.

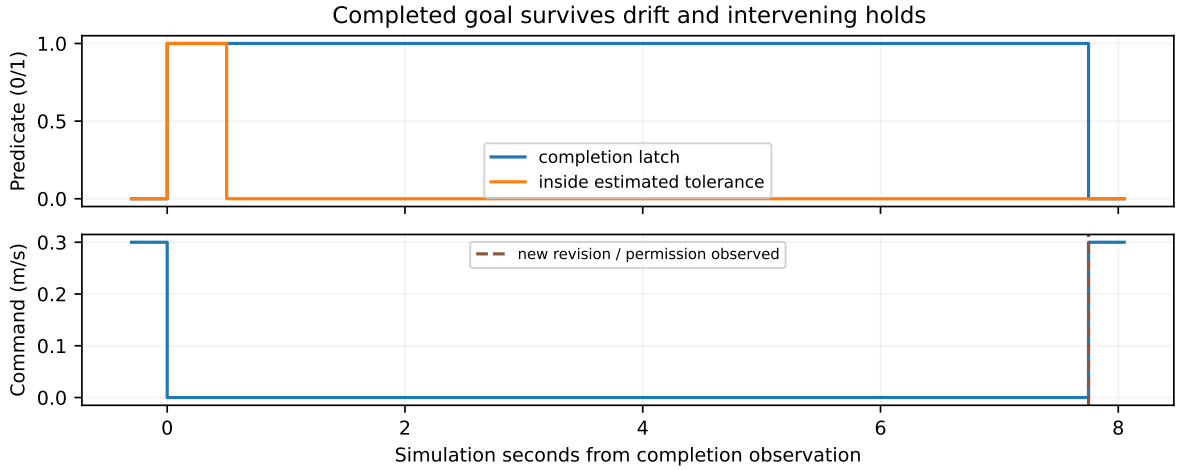
3 Results and analysis

The checked graph contains 92 controller states, 92 product states, and 7120 transitions. All 11 properties hold in this abstraction, and the compiled C++ function conforms over all 7120 contexts/inputs. All 9 unsafe mutations produce retained counterexamples, including early recovery, double counting, invented maturity, forgotten completion, and vacuous indefinite holds. The actual ROS controller passes 29 controlled cases; these use declared detector/pose/route inputs and do not measure detector accuracy.



Policy 7bc936680acc4199; implementation bd4b7647792d83fc. Discrete decisions / controlled ROS inputs; no physical-safety proof.

Ten endpoints at half-second spacing span only 4.5 seconds. The separate elapsed-time gate prevents release then; a later tick may release at five seconds if the latest decision remains fresh.



Policy 7bc936680acc4199; implementation bd4b7647792d83fc. Discrete decisions / controlled ROS inputs; no physical-safety proof.

Completion survives estimated drift, invalid guards, and fault recovery. A valid new revision permits a new mission only when the other navigation conditions hold. Tables and witnesses are in `properties.csv`, `cases.csv`, `mutations.csv`, and the saved proof.

4 Scope and course connection

The proof concerns finite decision logic under validated-input and time-abstraction assumptions. It does not prove delivery, scheduling, perception, learned detection, continuous clearance, or physical braking. Clock epochs are locally inferred; message types do not identify numerically coincident old messages after rewind. Timing and physical stopping remain separate measured trace claims. Historical M3/M4 cohorts are not retroactively covered by this proof. This insertion addresses architecture/assumptions, methods, implementation, and quantitative analysis in sections 3–6 of the course report template. Provided course materials are labelled Fall 2025; no current-year deadline or grading rule is inferred. Source provenance is documented in `docs/verification/supervisor_course_alignment.md`.

Tooling acknowledgment. Codex assistance was used for implementation and documentation. The correctness argument consists of the explicit model checks, negative controls, conformance, and recorded runtime evidence.